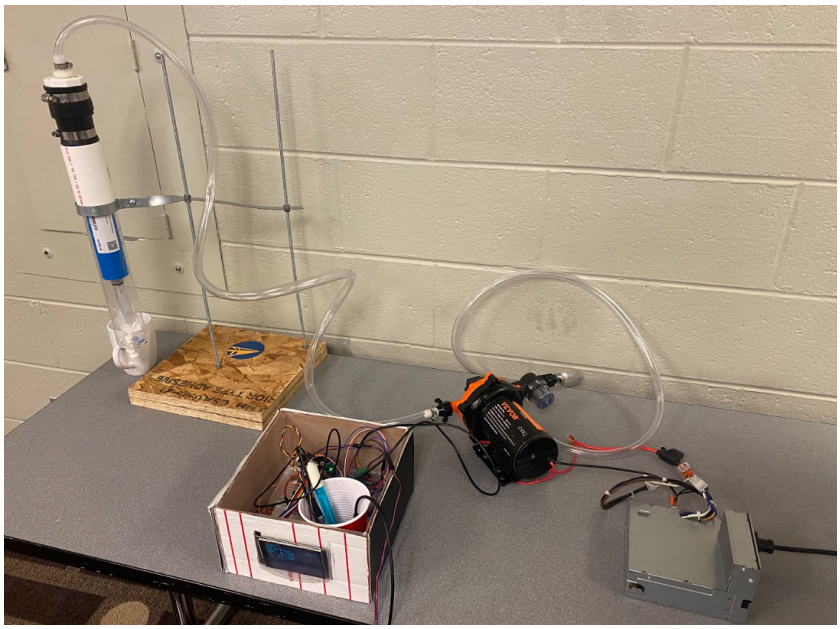
SCHOOLCRAFT ENGINEERING CLUB / COLLABORATIVE PROJECT

SMART WATER FILTRATION

Prototype Design, Sensor Monitoring & Integration



Completed prototype: pump, filter assembly, tubing, power hardware, and sensor enclosure. [P1]

ENGINEERING PROJECT REPORT

Schoolcraft Engineering Club
Collaborative team project

Prepared by
Zarraf Alam Fahmid

Abstract

The Schoolcraft Engineering Club developed a pump-driven water filtration prototype with electronic water-quality monitoring. The team integrated plumbing, power hardware, four sensor channels, and a color display through shared design, assembly, and troubleshooting. The prototype operated with lower water flow than expected. This report documents the hardware, software, integration challenges, and demonstrated outcome, followed by the complete project source code. [T1, P1-P3, R1]

Documented feature	Project scope
Sensor firmware	4 measurement channels; modular C++ architecture across 12 source files
Physical build	Pump, filter assembly, tubing, power hardware, and electronic display
Demonstration	Working filtration assembly with sensor display; flow below expectation

Report guide: System and components 2-4 | Hydraulic and electrical integration 5-7
Firmware 8 | Development challenges 9 | Results 10 | Code review 11
Conclusion and references 12 | Source-file register 13 | Full code 14-23

Prepared 25 October 2026 | Engineering prototype report

1 Project objectives and team responsibilities

The team combined fluid delivery, filtration hardware, embedded sensing, and a display in one demonstrator. Members worked across schematic design, coding, assembly, and debugging, with shared responsibility for making the subsystems operate together. [T1]

Engineering objectives

Objective	What the report documents
Deliver water through a filter	A physical pump-and-tubing assembly and the team account of filtration.
Monitor water-quality indicators	Repository implementations for four sensor channels and a display interface.
Integrate the subsystems	Mounted filtration hardware, a sensor enclosure, plumbing connections, and a displayed user interface.
Learn through iteration	Repeated equipment trials, fitting and material decisions, and work to combine the subsystems.



Figure 1. Demonstration setup showing the inlet container, pump, hose routing, mounted filter assembly, and output arrangement. [P2]

Outcome and reporting scope

The team reported that the prototype filtered water, but the flow was slower than expected. No timed flow measurements or paired inlet/outlet measurements are available. The results therefore describe operation of the prototype, rather than a measured treatment efficiency or drinking-water assessment. [T1]

2 System architecture and design scope

The project records contain three distinct elements: an early two-controller design, photographs of the assembled prototype, and source code for the sensor-display module. The concept includes flow, level, and pump-control functions; the available code implements four water-quality channels and a color TFT display. [D1-D3, P1-P3, R1]

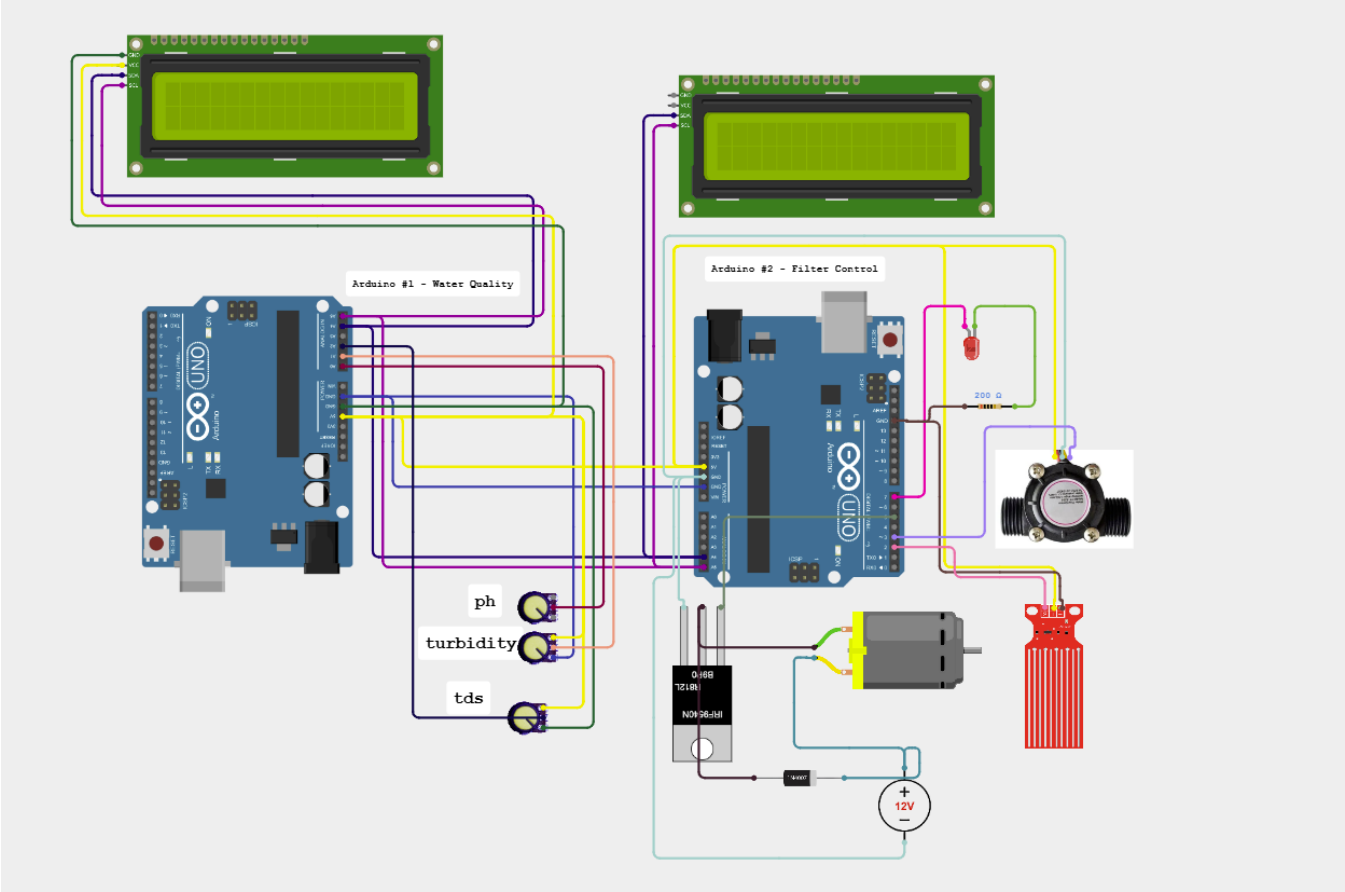


Figure 2. Original two-Arduino concept, reproduced unaltered. It includes two character displays, analog sensor stand-ins, a flow sensor, level sensor, and a motor-switching concept. This drawing represents the early concept. [D1]

Subsystem	Role and documented scope
Water path	Container, pump, tubing, fittings, and filtration assembly; physical build photographed.
Measurement path	Four sensor channels processed in the retrieved firmware.
Display path	Color TFT in the repository and prototype photos; character LCDs in the concept drawings.
Actuation path	Pump-switching components appear in planning material; pump-control firmware is not in the retrieved sensor-module source.

3 Components and configuration

The table identifies each component and its supporting record. Specifications marked as planned are design references; final installed part numbers and ratings were not recorded. [P1-P3, D4, R1]

Component group	Purpose	Evidence / configuration
Arduino UNO-class controller	Sensor acquisition and display coordination	UNO hardware appears in drawings and photos; repository has one main sketch.
pH probe and interface	Acidity/alkalinity indication	DFRobot_PH interface used in code; exact installed module revision not recorded.
TDS probe and interface	Conductivity-derived dissolved-solids estimate	Analog processing, median filtering, and temperature compensation in code.
Turbidity interface	Optical clarity indication	Analog input and normalized clarity calculation in code. SEN0189 is listed in the plan.
Temperature probe	Temperature measurement and compensation input	OneWire and DallasTemperature used in code; DS18B20 listed in the plan.
Color TFT display	Shows four sensor values or range errors	LCDWIKI SPI driver configured for ST7796S. Physical screen is photographed.
Water pump / motor	Drives flow through the plumbing	VEVOR pump is visible. The plan lists a 12 V, 60 PSI pump; actual rating is not confirmed here.
Filtration assembly	Physical water-treatment stage	Vertical assembly is photographed. The final cartridge specification and media sequence require the build BOM.
Tubing, fittings, adapters	Connects the hydraulic components	Transparent tubing, clamps, and connectors are visible. Materials and sizes were a major selection task.
Power supply / conversion	Provides the required electrical rails	Metal power hardware is visible; the team reports using a converter. Final voltage/current ratings are not documented.
Prototype wiring and enclosure	Organizes sensors and electronics	Breadboard, jumpers, interface modules, and a temporary enclosure are photographed.

Additional planned parts

The component plan also lists an N-channel MOSFET, a flyback diode, a gate pull-down resistor, an LED and resistor, a flow sensor, and a water-level device. The plan names an RO cartridge and carbon/cotton filtration options. These are planned items; their installation and final configuration are unconfirmed. [D4]

For a reproducible BOM, record manufacturer part numbers, pump and converter ratings, tube internal diameter and material, fitting threads and seals, filter cartridge model, and the final sensor/display revisions.

4 Hydraulic and mechanical integration

The team tested pump and motor options while resolving filter, fitting, and tubing compatibility. The objective was a usable flow path that could be assembled cleanly within the prototype layout. [T1]

Filter selection

Sediment and other filter options were considered alongside equipment availability and fitting compatibility. The final cartridge and media specification were not recorded in the supplied account. [T1]

Fittings, pipe size, and materials

Mismatched fittings and pipe sizes required repeated selection and layout adjustments. The team also considered water-contact material suitability when choosing tubing. Procurement, mechanical fit, and presentation were addressed together. [T1]

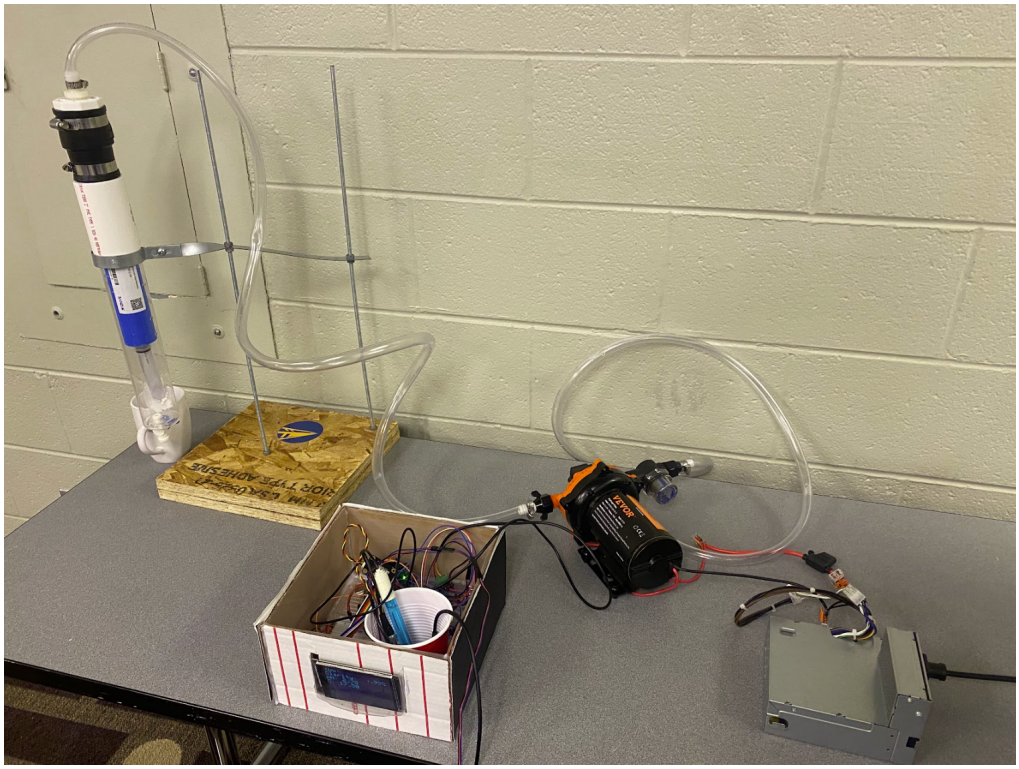


Figure 3. Integrated assembly showing the vertical filter support, hose loops, pump, electronic enclosure, and power hardware. [P1]

The flow-rate tradeoff

The completed assembly filtered water at a lower flow rate than expected. The cause was not isolated; pump operating point, filter resistance, plumbing losses, and supply behavior remain useful checks for a future revision. [T1]

5 Water-quality sensing circuit

The circuit design connects pH, TDS, turbidity, temperature, and display interfaces to the controller. Figure 4 records the design stage. The pin assignments below are taken from the source code. [D2, R1]

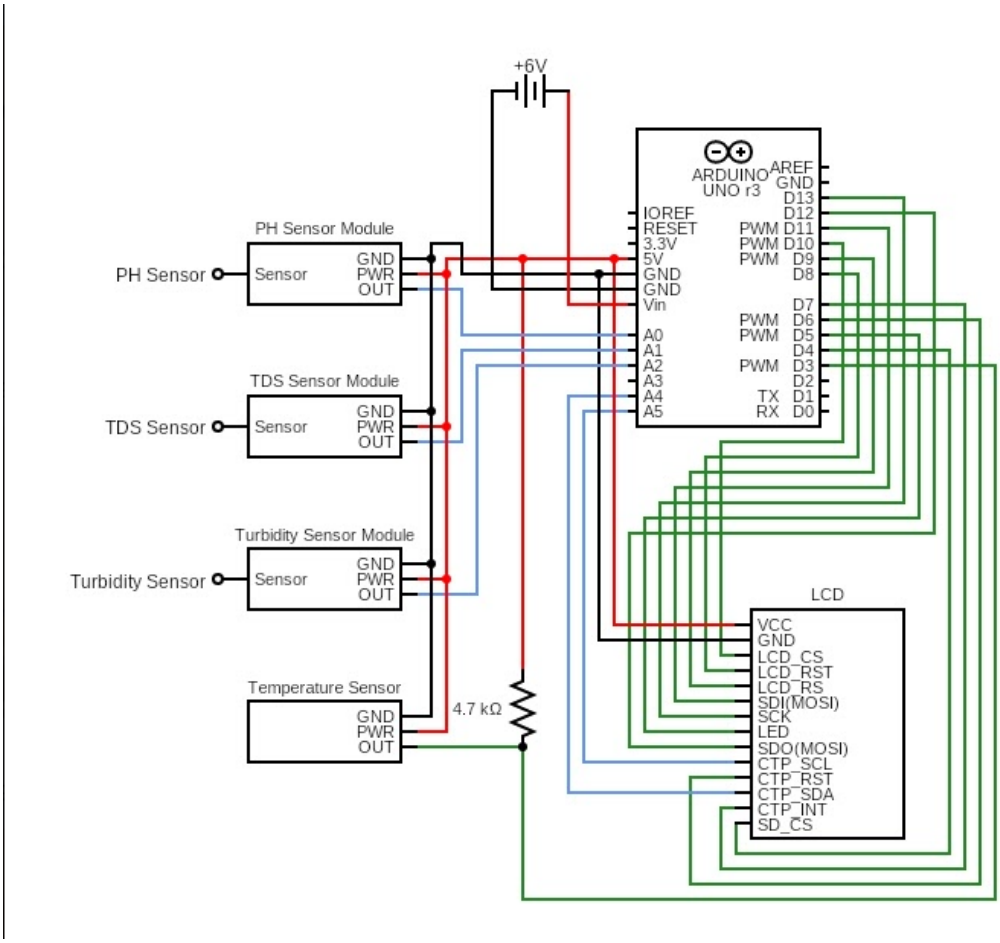


Figure 4. Water-quality sensing circuit from the design records. Supply and display connections differ across project revisions. [D2]

Sensor connections in the source code

Signal	Repository assignment	Interpretation
pH / TDS / turbidity	A0 / A1 / A2	Three analog sensor channels.
Temperature data	D3	OneWire temperature sensor interface.
Display interface	SPI-style pins and separate control signals	Color TFT configuration; differs from the character-LCD concept.

The drawing labels a 6 V source. Final supply voltages, power routing, and display wiring must be confirmed against the installed modules before this design is reproduced.

6 Flow, level, and pump-control concept

The early control concept includes flow and level inputs, a character LCD, and pump switching. This section records that concept; these control functions are not implemented in the available sensor-module code. [D1, D3, R1]

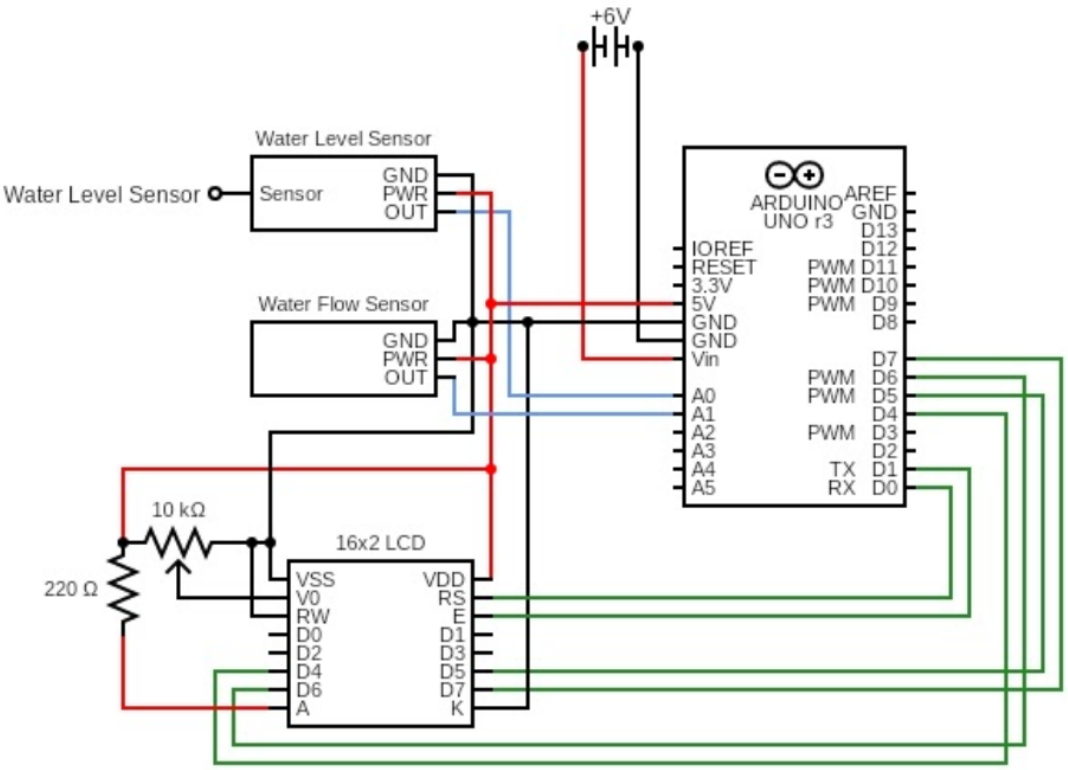


Figure 5. Early flow/level monitoring concept with a character LCD. [D3]

Pump and power integration

The team tested several motors and pumps and used a converter to integrate the power hardware. The component plan includes a MOSFET and flyback diode for motor switching. The final power and control configuration needs the following details for reproduction. [T1, D4]

Integration item	Documentation needed for the next build
Power conversion	Converter model, input/output voltages, current capacity, and loaded measurements.
Motor switching	Device part number, gate-drive arrangement, protection, and current rating.
Flow measurement	Sensor model, electrical interface, pulse calibration, and units.
Level protection	Device type, threshold behavior, and the actual pump-inhibit logic.

7 Firmware and sensor processing

The 12-file C++ implementation separates four sensor classes, a common sensor interface, a TFT display class, and the main sketch. This modular structure separates measurement processing from presentation. Source snapshot: `2fe374ec203a`. [R1]

Module	Responsibility
Main sketch	Creates the four sensor objects and display; initializes interfaces; reads and displays values in a repeated loop.
Sensor base	Defines the shared initialization interface.
TemperatureSensor	Requests a OneWire temperature conversion and retrieves Celsius/Fahrenheit readings.
PhSensor	Converts the analog reading to millivolts and calls the DFRobot pH conversion routine with temperature.
TDSSensor	Uses a 30-sample median and temperature compensation to estimate TDS.
TurbiditySensor	Normalizes analog sensor voltage against a 4.21 V reference to display clarity.
SensorModuleLCDScreen	Initializes the SPI TFT, formats four rows, and displays range errors.

Implemented pin assignments

Function	Assignment
Analog sensors	pH A0; TDS A1; turbidity A2
Temperature	D3
TFT control	CS D10; reset D8; register-select D9; backlight/LED D5
TFT data/clock	SDI D11; SDO D12; clock D13
Other defined display pins	Touch reset D6, touch interrupt D7, SD-card CS D4; these are not used by the shown loop.

Loop behavior

The loop acquires temperature, TDS, turbidity, and pH, then updates the four display values and waits one second. Temperature feeds the pH and TDS conversions; the full loop determines the actual sample and display cadence. [R1]

> The displayed clarity percentage is a normalized sensor-voltage indicator, not a measured contaminant-removal percentage or turbidity in NTU. Calibration and paired samples are required for quantitative treatment results. [R1]

8 Development challenges and responses

The team worked through equipment compatibility, mechanical packaging, and software integration to produce the demonstrator. The table summarizes the reported development experience. [T1]

Challenge	Team response
Filter selection	Compared sediment and other filter options while assessing equipment compatibility.
Pump and motor selection	Purchased and tested different devices to reach a usable configuration.
Fittings and tubing	Worked through connector and pipe-size mismatches; considered material suitability for water contact.
Software integration	Combined sensor and display functions and worked through problems getting the subsystems to operate together.
Physical packaging	Adjusted the plumbing and electronics layout for a coherent demonstration setup.
Flow performance	Achieved filtration with lower throughput than expected; the shortfall remained a development limitation.

Engineering outcome

The team turned separately sourced hardware and software into an assembled filtration demonstration. The work required practical tradeoffs across electrical supply, mechanical interfaces, sensor behavior, and layout, with slower flow retained as a clear performance limitation.

9 Demonstration results and limitations

The photographs show the assembled filtration hardware and electronic display. The team reported successful operation with slower-than-expected flow. The table separates these observations from measurements that were not recorded. [T1, P1-P3]

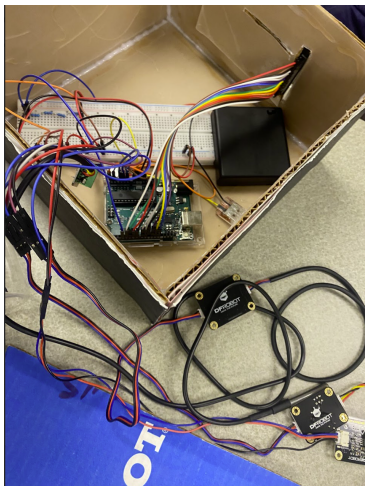


Figure 6a. Prototype electronics, breadboard wiring, controller, and sensor interfaces. [P3]

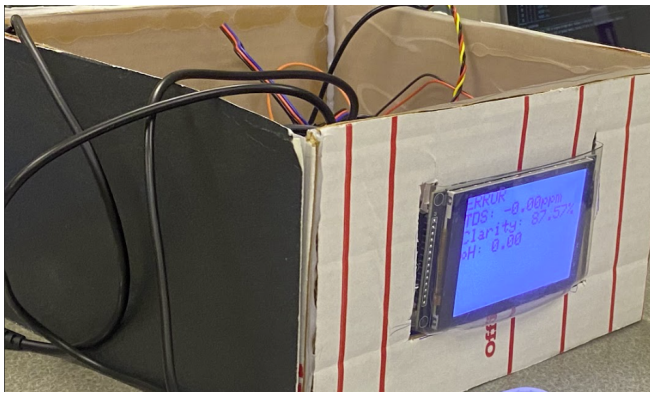


Figure 6b. Display during testing, showing sensor values and an ERROR indication. [P3]



Figure 7. Additional display capture. The original resolution is insufficient for reliable transcription. [P3]

Result category	Supported conclusion
Physical prototype	Pump, tubing, filtration assembly, sensor electronics, and display were assembled.
Software implementation	Four measurement channels and a TFT display are present in the retrieved source.
Filtration outcome	The team reports water was cleaned; no paired quantitative treatment result accompanies the report.
Throughput	Reported as lower than expected; neither the target nor a measured L/min value was supplied.
Water quality	Display values demonstrate the interface; calibration and laboratory evidence are needed for validated quality claims.

10 Software review and build requirements

Inspection of the archived code identified the following issues for a future revision. They are separate from the team-reported development challenges in Section 8. No fixes or bench-test results are claimed here. [R1]

Review finding	Why it matters / proposed action
Temperature interface lifetime	TemperatureSensor::begin creates the OneWire object as a local variable and passes its address into DallasTemperature. The retained sensor interface can outlive that local object. Keep the OneWire instance alive for the lifetime of the sensor.
TDS sample cadence	The code checks a nominal 40 ms interval, but calls sampling once per main loop and then delays one second. Schedule acquisition independently if faster sampling is required.
TDS buffer readiness	The median uses the full 30-slot buffer before a full set of fresh samples is established. Track fill count and show a warming-up state.
Missing return value	TDSSensor::readTdsValue is declared to return float but does not return one. The current caller uses getTds afterward; align the declared interface with the intended behavior.
Display bounds checking	The displayData argument is a pointer, so sizeof on that argument does not give the caller’s array length. Pass an explicit count or a fixed-size reference and enforce the four-slot capacity.
pH range and calibration state	Conversion is conditional on temperature being between 5 and 60 °C. Add explicit validity flags and a deliberate response to an invalid temperature or uncalibrated probe.
Initialization consistency	The main setup initializes temperature, pH, TDS, and display but omits the turbidity begin method. Make initialization explicit across modules.

Build environment

The source depends on LCDWIKI_GUI, LCDWIKI_SPI, DFRobot_PH, OneWire, and DallasTemperature, including headers brought in by those libraries. Board-core and library versions are not pinned in the supplied root files. A reproducible release should record versions, calibration state, wiring revision, and the exact sketch directory. [R1]

Verification status: source inspection only. Compilation, dependency compatibility, and sensor accuracy have not been verified for this archived revision.

11 Conclusion and references

The team delivered a physical filtration demonstrator combining pump-driven flow, four-channel sensor firmware, and a color display. Equipment trials and shared troubleshooting addressed plumbing, power, packaging, and software integration. The result was a working demonstration with slower-than-expected flow, supported by photographs, circuit records, and the source code included here. [T1, P1-P3, R1]

Collaboration statement

This report presents a **Schoolcraft Engineering Club collaborative project**. Schematic design, coding, component selection, assembly, integration, and troubleshooting were shared across team members. [T1]

ID	Source and use
T1	Project account supplied by Zarraf Alam Fahmid: shared work, equipment trials, filter/fitting/tubing decisions, software integration, and qualitative outcome.
R1	Ce3Pi0 / Water-Filter-Sensor-Module GitHub repository. Retrieved for this report; immutable commit recorded below. Firmware architecture and review findings.
P1	Screenshots 2026-05-12 183053 and 2026-10-23 200338 / 200352. Physical prototype and plumbing arrangement; overlapping views.
P2	Screenshot 2026-10-25 211315. Demonstration setup with container, pump, filter assembly, and tubing.
P3	Screenshots 2026-05-12 183120 / 183235 and 2026-10-25 211257. Electronics and display captures.
D1	Screenshot 2026-10-23 202148; exact selected source image. Two-controller wiring concept, retained unaltered in Section 2.
D2-D3	Screenshots 2026-10-23 200241 and 200229. Separate sensor/display and flow/level circuit drawings.
D4	Screenshot 2026-10-23 202036. Component planning table; specifications require confirmation against the installed BOM.

Repository: https://github.com/Ce3Pi0/Water-Filter-Sensor-Module

Commit: 2fe374ec203a28442bd69c5f18ae49f41ea41d89
Snapshot inspected: 25 October 2026. Repository commit date: 8 December 2025. The snapshot is not asserted to be the final demonstration firmware.

Appendix Source-file register

All 12 source files are reproduced in full in Appendices A-F, beginning on page 14, from repository commit 2fe374ec203a. Line numbers restart for each file; unnumbered continuation lines are visual wraps. The source logic and comments are preserved, including the issues discussed in Section 10. [R1]

Source file	Role	Lines
PhSensor.cpp	pH voltage acquisition and conversion	23
PhSensor.h	pH class declaration	22
Sensor.h	Abstract sensor initialization interface	11
SensorModuleLCDScreen.cpp	Display initialization, formatting, and range checks	103
SensorModuleLCDScreen.h	TFT model, display bounds, and class declaration	45
TDSSensor.cpp	TDS sampling, median filtering, and conversion	59
TDSSensor.h	TDS class, buffer, and constants	25
TemperatureSensor.cpp	OneWire/DallasTemperature setup and acquisition	32
TemperatureSensor.h	Temperature class declaration	21
TurbiditySensor.cpp	Analog acquisition and normalized clarity	28
TurbiditySensor.h	Turbidity class and voltage references	22
Water-Filter-Sensor-Module.ino	Sketch setup, acquisition order, display update, and delay	73

Full-code appendix guide

Appendix A - Main Arduino sketch: pages 14-15
Appendix B - Sensor interface and pH: pages 16
Appendix C - Temperature sensor: pages 17
Appendix D - TDS sensor: pages 18-19
Appendix E - Turbidity sensor: pages 20
Appendix F - TFT display: pages 21-23

Scope boundaries

The repository has four sensor channels and one display path. It does not contain an implemented flow/level controller, pump state machine, or two-controller protocol. To publish the entire project as a reproducible build, retain those additional records with the hydraulic BOM, calibration files, and measured test results.

Recommended release package: a pinned firmware revision, dependency versions, native schematics, an as-built wiring map, a hydraulic BOM, calibration records, and repeated flow and water-quality test logs.

Appendix A Main Arduino sketch

Original project source | commit 2fe374ec203a | listing 1 of 2
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

FILE: Water-Filter-Sensor-Module.ino

```
#include "SensorModuleLCDScreen.h"
#include "PhSensor.h"
#include "TurbiditySensor.h"
#include "TDSSensor.h"
#include "TemperatureSensor.h"

//Analog Pins
#define PH_PIN A0
#define TDS_PIN A1
#define TURBIDITY_PIN A2
#define TEMP_PIN 3

//Digital Pins
#define LCD_CS 10
#define LCD_RST 8
#define LCD_RS 9
#define SDI 11
#define SCK 13
#define LED 5
#define SDO 12
#define CTP_RST 6
#define CTP_INT 7
#define SD_CS 4

const int INPUT_COUNT = 4;

// LCD screen
SensorModuleLCDScreen lcd(LCD_CS, LCD_RS, SDO, SDI, LCD_RST, SCK, LED, INPUT_COUNT);

// Temperature Sensor
TemperatureSensor temperatureSensor(TEMP_PIN);
// PH Sensor
PhSensor phSensor(PH_PIN);
// Turbidity Sensor
TurbiditySensor turbiditySensor(TURBIDITY_PIN);
// TDS Sensor
```

Appendix A Main Arduino sketch - continued

Original project source | commit 2fe374ec203a | listing 2 of 2
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

```
37  TDSSensor tdsSensor(TDS_PIN);
38
39  void setup() {
40    lcd.begin();
41    temperatureSensor.begin();
42    phSensor.begin();
43    tdsSensor.begin();
44  }
45
46  void loop() {
47    lcd.clearScreen();
48
49    // Temp sensor interactions
50    temperatureSensor.readTemp();
51    float tempValueC = temperatureSensor.getTempC();
52
53    // TDS sensor interactions
54    tdsSensor.getTdsSamples();
55    tdsSensor.readTdsValue(tempValueC);
56    float tdsValue = tdsSensor.getTds();
57
58    // Turbidity sensor interactions
59    turbiditySensor.readTurbidity();;
60    turbiditySensor.getClarityPercentage();
61    float turbidityValue = turbiditySensor.getClarity();
62
63    // PH sensor interactions
64    phSensor.getPhVoltage();
65    phSensor.readPh(tempValueC);
66    float phValue = phSensor.getPh();
67
68    float data[INPUT_COUNT] = {tempValueC, tdsValue, turbidityValue, phValue};
69
70    lcd.displayData(data);
71
72    delay(1000);
73  }
```

Appendix B Sensor interface and pH

Original project source | commit 2fe374ec203a | listing 1 of 1
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

FILE: Sensor.h

```
 1  #ifndef SENSOR_H
 2  #define SENSOR_H
 3
 4  class Sensor {
 5    protected:
 6      unsigned short sensorPin;
 7    public:
 8      virtual void begin() = 0;
 9  };
10
11  #endif
```

FILE: PhSensor.h

```
 1  #ifndef PHSENSOR_H
 2  #define PHSENSOR_H
 3
 4  #include "Sensor.h"
 5  #include "DFRobot_PH.h"
 6
 7
 8  class PhSensor:protected Sensor {
 9    private:
10      DFRobot_PH sensor;
11      float phVoltage, phValue;
12
13    public:
14      PhSensor(unsigned int);
15
16      void begin();
17      void getPhVoltage();
18      void readPh(float);
19      float getPh();
20  };
21
22  #endif
```

FILE: PhSensor.cpp

```
 1  #include "DFRobot_PH.h"
 2  #include "PhSensor.h"
 3
 4
 5  PhSensor::PhSensor(unsigned int sensorPin) {
 6    this->sensorPin = sensorPin;
 7  }
 8
 9  void PhSensor::begin() {
10    this->sensor.begin();
11  }
12
13  void PhSensor::getPhVoltage() {
14    if (this->sensorPin <= A0) this->phVoltage = analogRead(this->sensorPin) / 1024.0 * 5000;  // Read
     the phVoltage
15  }
16
17  void PhSensor::readPh(float temp) {
18    if (temp >= 5 && temp <= 60) this->phValue = this->sensor.readPH(phVoltage, temp);  // Convert
    phVoltage to pH with temperature compensation
19  }
20
21  float PhSensor::getPh() {
22    return this->phValue;
23  }
```

Appendix C Temperature sensor

Original project source | commit 2fe374ec203a | listing 1 of 1
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

FILE: TemperatureSensor.h

```h
#include <DallasTemperature.h>

#ifndef TEMPERATURESENSOR_H
#define TEMPERATURESENSOR_H

#include "Sensor.h"

class TemperatureSensor:protected Sensor {
    private:
        DallasTemperature sensor;
        float tempValueC, tempValueF;
    public:
        TemperatureSensor(unsigned int);

        void begin();
        void readTemp();
        float getTempC();
        float getTempF();
};

#endif
```

FILE: TemperatureSensor.cpp

```cpp
#include <OneWire.h>
#include <DallasTemperature.h>

#include "TemperatureSensor.h"

TemperatureSensor::TemperatureSensor(unsigned int sensorPin){
    this->sensorPin = sensorPin;
}

void TemperatureSensor::begin(){
    // Setup a oneWire instance to communicate with any OneWire devices
    OneWire oneWire(this->sensorPin);
    // Pass our oneWire reference to Dallas Temperature sensor
    DallasTemperature tempSensor(&oneWire);
    this->sensor = tempSensor;

    this->sensor.begin();
}

void TemperatureSensor::readTemp(){
  this->sensor.requestTemperatures();
  this->tempValueC = this->sensor.getTempCByIndex(0);
  this->tempValueF = this->sensor.getTempFByIndex(0);
}

float TemperatureSensor::getTempC(){
  return this->tempValueC;
}

float TemperatureSensor::getTempF(){
  return this->tempValueF;
}
```

Appendix D TDS sensor

Original project source | commit 2fe374ec203a | listing 1 of 2
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

FILE: TDSSensor.h

```h
#ifndef TDSSENSOR_H
#define TDSSENSOR_H

#include "Sensor.h"

class TDSSensor:protected Sensor {
    private:
        static const int SCOUNT = 30;
        const float VREF = 5.0f;

        int analogBuffer[SCOUNT];  // store the analog value in the array, read from ADC
        int analogBufferTemp[SCOUNT];
        int analogBufferIndex = 0, copyIndex = 0;
        float averageVoltage = 0.0f, tdsValue = 0.0f;
    public:
        TDSSensor(unsigned int);

        void begin();
        void getTdsSamples();
        float readTdsValue(float);
        int getMedianNum(int[], int);
        float getTds();
};

#endif
```

FILE: TDSSensor.cpp

```cpp
#include <Arduino.h>
#include "TDSSensor.h"

TDSSensor::TDSSensor(unsigned int sensorPin) {
    this->sensorPin = sensorPin;
}

void TDSSensor::begin(){
    pinMode(this->sensorPin, INPUT);
}

void TDSSensor::getTdsSamples() {
  static unsigned long analogSampleTimepoint = millis();
  if (millis() - analogSampleTimepoint > 40U) {  //every 40 milliseconds,read the analog value from the ADC
    analogSampleTimepoint = millis();
    this->analogBuffer[this->analogBufferIndex] = analogRead(this->sensorPin);  //read the analog value and store into the buffer
    this->analogBufferIndex++;
    if (this->analogBufferIndex == this->SCOUNT)
```

Appendix D TDS sensor - continued

Original project source | commit 2fe374ec203a | listing 2 of 2
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

```
      this->analogBufferIndex = 0;
  }
}

float TDSSensor::readTdsValue(float temperature) {
  for (this->copyIndex = 0; this->copyIndex < this->SCOUNT; this->copyIndex++)
    this->analogBufferTemp[this->copyIndex] = this->analogBuffer[this->copyIndex];

  this->averageVoltage = this->getMedianNum(this->analogBufferTemp, this->SCOUNT) *
(float)this->VREF / 1024.0;

// read the analog value more stable by the median filtering algorithm, and convert to voltage value
  float compensationCoefficient = 1.0 + 0.02 * (temperature - 25.0);
                                                                        //temperature
compensation formula: fFinalResult(25^C) = fFinalResult(current)/(1.0+0.02*(fTP-25.0));
  float compensationVoltage = this->averageVoltage / compensationCoefficient;
                                                                              //temperature
compensation
  tdsValue = (133.42 * compensationVoltage * compensationVoltage * compensationVoltage - 255.86 *
compensationVoltage * compensationVoltage + 857.39 * compensationVoltage) * 0.5;  //convert voltage
value to tds value
}

int TDSSensor::getMedianNum(int bArray[], int iFilterLen) {
  int bTab[iFilterLen];
  for (byte i = 0; i < iFilterLen; i++)
    bTab[i] = bArray[i];
  int i, j, bTemp;
  for (j = 0; j < iFilterLen - 1; j++) {
    for (i = 0; i < iFilterLen - j - 1; i++) {
      if (bTab[i] > bTab[i + 1]) {
        bTemp = bTab[i];
        bTab[i] = bTab[i + 1];
        bTab[i + 1] = bTemp;
      }
    }
  }
  if ((iFilterLen & 1) > 0)
    bTemp = bTab[(iFilterLen - 1) / 2];
  else
    bTemp = (bTab[iFilterLen / 2] + bTab[iFilterLen / 2 - 1]) / 2;
  return bTemp;
}

float TDSSensor::getTds() {
  if (this->tdsValue < 0.0f) {
    return -1.0f;
  }
  return this->tdsValue;
}
```

Appendix E Turbidity sensor

Original project source | commit 2fe374ec203a | listing 1 of 1
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

FILE: TurbiditySensor.h

```h
#ifndef TURBIDITYSENSOR_H
#define TURBIDITYSENSOR_H

#include "Sensor.h"

class TurbiditySensor:protected Sensor {
  private:
    const float MAX_CLARITY = 100.0f;
    const float VREF = 5.0f;
    const float CLEAN_WATER_VOLTAGE = 4.21f;
    float turbidity, clarity, voltage;
  public:
    TurbiditySensor(unsigned int);

    void begin();
    float turbidityToVoltage();
    void readTurbidity();
    void getClarityPercentage();
    float getClarity();
};

#endif
```

FILE: TurbiditySensor.cpp

```cpp
#include <Arduino.h>
#include "TurbiditySensor.h"

TurbiditySensor::TurbiditySensor(unsigned int sensorPin) {
    this->sensorPin = sensorPin;
}

void TurbiditySensor::begin(){
  pinMode(this->sensorPin, INPUT);
}

float TurbiditySensor::turbidityToVoltage()  {
  return this->turbidity * (this->VREF / 1024.0);
}

void TurbiditySensor::readTurbidity() {
    this->turbidity = analogRead(this->sensorPin);
}

void TurbiditySensor::getClarityPercentage() {
  this->voltage = this->turbidityToVoltage();
  this->clarity = this->voltage / this->CLEAN_WATER_VOLTAGE * 100.0f;
}

float TurbiditySensor::getClarity() {
  if (this->clarity > this->MAX_CLARITY) return -1.0f;
  return this->clarity;
}
```

Appendix F TFT display

Original project source | commit 2fe374ec203a | listing 1 of 3
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

FILE: SensorModuleLCDScreen.h

```
#include <LCDWIKI_SPI.h>

#ifndef SENSORMODULELCDSCREEN_H
#define SENSORMODULELCDSCREEN_H

#define LCD_MODEL ST7796S

class SensorModuleLCDScreen {
    private:
        const float MIN_TEMP_C = -55.0f, MAX_TEMP_C = 125.0f;
        const float MIN_TDS_VALUE = 0.0f;
        const float MIN_CLARITY = 0.0f, MAX_CLARITY = 100.0f;
        const float MIN_PH_VALUE = 0.0f, MAX_PH_VALUE = 14.0f;

        const unsigned int FONT_SIZE = 5;
        const unsigned int BLACK = 0;
        const unsigned int TEXT_COLOR = 1000;
        LCDWIKI_SPI* lcd;
        unsigned int inputCount;
        float data[4];

        void displayTempC(float);
        void displayTds(float);
        void displayClarity(float);
        void displaypH(float);

        void clearTempC(float);
        void clearTds(float);
        void clearClarity(float);
        void clearpH(float);

        enum ROWS{
            FIRST = 0,
            SECOND = 45,
            THIRD = 90,
            FOURTH = 135
        };
    public:
        SensorModuleLCDScreen(unsigned int, unsigned int, unsigned int, unsigned int, unsigned int,
    unsigned int, unsigned int, unsigned int);
        void begin();
        void clearScreen();
        void displayData(float[]);
};

#endif
```

FILE: SensorModuleLCDScreen.cpp

```
#include <Arduino.h>
```

Appendix F TFT display - continued

Original project source | commit 2fe374ec203a | listing 2 of 3
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

```
#include <LCDWIKI_GUI.h>
#include <LCDWIKI_SPI.h>

#include <lcd_spi_registers.h>
#include <mcu_spi_magic.h>

#include "SensorModuleLCDScreen.h"

SensorModuleLCDScreen::SensorModuleLCDScreen(unsigned int LCD_CS, unsigned int LCD_RS, unsigned int SDO, unsigned int SDI, unsigned int LCD_RST, unsigned int SCK, unsigned int LED, unsigned int inputCount){
    LCDWIKI_SPI* lcd = new LCDWIKI_SPI(LCD_MODEL, LCD_CS, LCD_RS, SDO, SDI, LCD_RST, SCK, LED);
    this->lcd = lcd;
    this->inputCount = inputCount;
}

void SensorModuleLCDScreen::begin(){
    this->lcd->Init_LCD();
    this->lcd->Set_Rotation(1); // Horizontal Display
    this->lcd->Set_Text_Mode(true);
    this->lcd->Set_Text_Size(this->FONT_SIZE);
    this->lcd->Fill_Screen(this->BLACK); // Background Color
}

void SensorModuleLCDScreen::clearScreen(){
    if (sizeof(data) / sizeof(float) < this->inputCount) return;

    this->lcd->Set_Text_colour(this->BLACK);

    this->clearTempC(data[0]);
    this->clearTds(data[1]);
    this->clearClarity(data[2]);
    this->clearpH(data[3]);

    this->lcd->Set_Text_colour(this->TEXT_COLOR);
}

void SensorModuleLCDScreen::displayData(float data[]){
    if (sizeof(data) / sizeof(float) > this->inputCount) return;

    for (int i = 0; i < this->inputCount; i++)
        this->data[i] = data[i];

    this->displayTempC(data[0]);
    this->displayTds(data[1]);
    this->displayClarity(data[2]);
    this->displaypH(data[3]);
}

void SensorModuleLCDScreen::displayTempC(float tempValue){
    if (tempValue < this->MIN_TEMP_C || tempValue > this->MAX_TEMP_C){
        this->lcd->Print_String("ERROR", 0, SensorModuleLCDScreen::FIRST);
    } else {
```

Appendix F TFT display - continued

Original project source | commit 2fe374ec203a | listing 3 of 3
Source: Ce3Pi0 / Water-Filter-Sensor-Module [R1]

```
        String output = "C: " + String(tempValue);
        this->lcd->Print_String(output, 0, SensorModuleLCDScreen::FIRST);
    }
}

void SensorModuleLCDScreen::displayTds(float tdsValue) {
    if (tdsValue < this->MIN_TDS_VALUE) {
        this->lcd->Print_String("ERROR", 0, SensorModuleLCDScreen::SECOND);
    } else {
        String output = "TDS: " + String(tdsValue) + "ppm\n";
        this->lcd->Print_String(output, 0, SensorModuleLCDScreen::SECOND);
    }
}

void SensorModuleLCDScreen::displayClarity(float clarity) {
    if (clarity < this->MIN_CLARITY || clarity > this->MAX_CLARITY) {
        this->lcd->Print_String("ERROR", 0, SensorModuleLCDScreen::THIRD);
    } else {
        String output = "Clarity: " + String(clarity) + '%';
        this->lcd->Print_String(output, 0, SensorModuleLCDScreen::THIRD);
    }
}

void SensorModuleLCDScreen::displaypH(float phValue) {
    if (phValue < this->MIN_PH_VALUE || phValue > this->MAX_PH_VALUE) {
        this->lcd->Print_String("ERROR", 0, SensorModuleLCDScreen::FOURTH);
    } else {
        String output = "pH: " + String(phValue);
        this->lcd->Print_String(output, 0, SensorModuleLCDScreen::FOURTH);
    }
}

void SensorModuleLCDScreen::clearTempC(float tempValue) {
    String tempOutput = "   " + String(tempValue);
    this->lcd->Print_String(tempOutput, 0, SensorModuleLCDScreen::FIRST);
}

void SensorModuleLCDScreen::clearTds(float tdsValue) {
    String tdsOutput = "     " + String(tdsValue) + "ppm\n";
    this->lcd->Print_String(tdsOutput, 0, SensorModuleLCDScreen::SECOND);
}

void SensorModuleLCDScreen::clearClarity(float clarity) {
    String clarityOutput = "         " + String(clarity) + '%';
    this->lcd->Print_String(clarityOutput, 0, SensorModuleLCDScreen::THIRD);
}

void SensorModuleLCDScreen::clearpH(float phValue) {
    String phOutput = "    " + String(phValue);
    this->lcd->Print_String(phOutput, 0, SensorModuleLCDScreen::FOURTH);
}
```